



PRODUCCIÓN	REGLAS SEMÁNTICAS
$P \rightarrow D$	$dir \leftarrow 0$
$D \rightarrow T L ; D$	$L.tipo = T.tipo$
$D \rightarrow \text{def } T \text{ id } (F) \{ S \} D$	Si no $PilaTs.bottom().existe(id)$ Entonces $PilaTs.push(nuevaTs())$ $PilaDir.push(dir)$ $dir \leftarrow 0$ $tipoReturnFunc \leftarrow T.tipo$ $label(id)$ $S.code$ $PilaTs.pop()$ $PilaTs.bottom().add(id, -1, T.tipo, 'func', F.lista)$ Sino $error(\text{El id ya fue declarado anteriormente})$ Fin si
$D \rightarrow \varepsilon$	
$T \rightarrow B A$	$T.tipo = A.tipo$ $A.base = B.tipo$
$T \rightarrow \text{register } \{ D \}$	$PilaTs.push(nuevaTs())$ $PilaDir.push(dir)$ $dir \leftarrow 0$ $*ts = PilaTs.top()$ $PilaTs.pop()$ $T.tipo = tablaTipos.add('struct', *ts)$ $dir \leftarrow PilaDir.top()$ $PilaDir.pop()$
$B \rightarrow \text{int}$	$B.tipo = tablaTipos.getId('int')$
$B \rightarrow \text{float}$	$B.tipo = tablaTipos.getId('float')$
$A \rightarrow [\text{num}] A_1$	Si $num.tipo = \text{int}$ Entonces Si $num.val > 0$ Entonces $A.tipo = tablaTipos.add(num.val, A_1.tipo, 'array')$ $A_1.base = A.base$ Sino $error(\text{El índice debe ser mayor a cero})$ Fin Si Sino $error(\text{El índice debe ser entero})$ Fin Si
$A \rightarrow \varepsilon$	$A.tipo = A.base$

$L \rightarrow L_1, id$	$L_1.tipo = L.tipo$ Si $noPilaTs.top().existe(id)$ Entonces $PilaTs.top().add(id, dir, L.tipo, 'var')$ $dir \leftarrow dir + tablaTipos.getTam(L.tipo)$ Sino $error(La\ variable\ ya\ fue\ declarada)$ Fin Si
$L \rightarrow id$	Si $noPilaTs.top().existe(id)$ Entonces $PilaTs.top().add(id, dir, L.tipo, 'var')$ $dir \leftarrow dir + tablaTipos.getTam(L.tipo)$ Sino $error(La\ variable\ ya\ fue\ declarada)$ Fin Si
$F \rightarrow G$	$F.lista = G.lista$
$F \rightarrow \varepsilon$	$F.lista = nulo$
$G \rightarrow G_1, T id$	Si $noPilaTs.top().existe(id)$ Entonces $PilaTs.top().add(id, dir, L.tipo, 'arg')$ $dir \leftarrow dir + tablaTipos.getTam(L.tipo)$ Sino $error(Argumento\ duplicado)$ Fin Si $G.lista = G_1.lista$ $G.lista.add(T.tipo)$
$G \rightarrow T id$	Si $noPilaTs.top().existe(id)$ Entonces $PilaTs.top().add(id, dir, L.tipo, 'var')$ $dir \leftarrow dir + tablaTipos.getTam(L.tipo)$ Sino $error(La\ variable\ ya\ fue\ declarada)$ Fin Si $G.lista = nuevaLista()$ $G.lista.add(T.tipo)$
$S \rightarrow S_1 S_2$	
$S \rightarrow \{ S_1 \}$	
$S \rightarrow \text{break};$	$genCode('goto' PilaLabelFalse.top())$
$S \rightarrow \text{return } E;$	Si $tipoReturnFunc = E.tipo$ Entonces $genCode('return' E.dir)$ Sino $error(Tipo\ de\ retorno\ incorrecto)$ Fin Si
$S \rightarrow id = E ;$	Si $PilaTs.top().existe(id)$ Entonces $tipo = PilaTs.top().getTipo(id)$ Si $compatibles(tipo, E.tipo)$ Entonces $\alpha_1 = reducir(E.dir, E.tipo, tipo)$ $genCode(id '=' \alpha_1)$ Sino $error(tipos\ incompatibles)$ Fin Si Si $PilaTs.bottom().existe(id)$ Entonces $tipo = PilaTs.bottom().getTipo(id)$

	Si compatibles(tipo, E.tipo) Entonces $\alpha_1 = \text{reducir}(E.\text{dir}, E.\text{ipo}, \text{tipo})$ genCode(id '=' α_1) Sino error(tipos incompatibles) Fin Si Sino error("El id no fue declarado") Fin Si
$S \rightarrow C = E ;$	Si compatibles(C.tipo, E.tipo) Entonces $\alpha_1 = \text{reducir}(E.\text{dir}, E.\text{ipo}, C.\text{tipo})$ genCode(C.base '['C.dir']' '=' α_1) Sino error(tipos incompatibles) Fin Si
$S \rightarrow Z = E ;$	Si compatibles(Z.tipo, E.tipo) Entonces $\alpha_1 = \text{reducir}(E.\text{dir}, E.\text{ipo}, Z.\text{tipo})$ genCode(Z.base '['Z.dir']' '=' α_1) Sino error(tipos incompatibles) Fin Si
$S \rightarrow \text{if}(E) S_1$	PilaLabelTrue.push(newLabel()) PilaLabelFalse.push(newLabel()) genCode('if' E.dir 'goto' PilaLabelTrue.top()) genCode('goto' PilaLabelFalse.top()) label(PilaLabelTrue.top()) $S_1.\text{code}$ label(PilaLabelFalse.top()) PilaLabelTrue.pop() PilaLabelFalse.pop()
$S \rightarrow \text{if}(E) S_1 \text{ else } S_2$	PilaLabelTrue.push(newLabel()) PilaLabelFalse.push(newLabel()) PilaLabelNext.push(newLabel()) genCode('if' E.dir 'goto' PilaLabelTrue.top()) genCode('goto' PilaLabelFalse.top()) label(PilaLabelTrue.top()) $S_1.\text{code}$ genCode('goto' PilaLabelNext.top()) label(PilaLabelFalse.top()) $S_2.\text{code}$ label(PilaLabelNext.top()) PilaLabelTrue.pop() PilaLabelFalse.pop() PilaLabelNext.pop()
$S \rightarrow \text{while}(E) S_1$	PilaLabelNext.push(newLabel()) PilaLabelTrue.push(newLabel()) PilaLabelFalse.push(newLabel()) label(PilaLabelNext.top()) E.code genCode('if' E.dir 'goto' PilaLabelTrue.top()) genCode('goto' PilaLabelFalse.top()) label(PilaLabelTrue.top()) $S_1.\text{code}$ genCode('goto' PilaLabelNext.top())

	label(PilaLabelFalse.top()) PilaLabelTrue.pop() PilaLabelFalse.pop() PilaLabelNext.pop()
$S \rightarrow \text{for}(S_1; E; S_2) S_3$	PilaLabelNext.push(newLabel()) PilaLabelTrue.push(newLabel()) PilaLabelFalse.push(newLabel()) PilaLabelInc.push(newLabel()) S_1 .code label(PilaLabelNext.top()) E .code genCode('if' E .dir 'goto' PilaLabelTrue.top()) genCode('goto' PilaLabelFalse.top()) label(PilaLabelInc.top()) S_2 .code genCode('goto' PilaLabelInc.top()) label(PilaLabelTrue.top()) S_3 .code genCode('goto' PilaLabelInc.top()) label(PilaLabelFalse.top()) PilaLabelTrue.pop() PilaLabelFalse.pop() PilaLabelNext.pop() PilaLabelInc.pop()
$E \rightarrow E_1 + E_2$	Si compatibles(E_1 .tipo, E_2 .tipo) Entonces E .dir = nuevaTemporal() E .tipo = max(E_1 .tipo, E_2 .tipo) α_1 = ampliar(E_1 .dir, E_1 .tipo, E .tipo) α_2 = ampliar(E_2 .dir, E_2 .tipo, E .tipo) genCode(E .dir '=' α_1 '+' α_2) Sino error(tipos incompatibles) Fin Si
$E \rightarrow E_1 - E_2$	Si compatibles(E_1 .tipo, E_2 .tipo) Entonces E .dir = nuevaTemporal() E .tipo = max(E_1 .tipo, E_2 .tipo) α_1 = ampliar(E_1 .dir, E_1 .tipo, E .tipo) α_2 = ampliar(E_2 .dir, E_2 .tipo, E .tipo) genCode(E .dir '=' α_1 '-' α_2) Sino error(tipos incompatibles) Fin Si
$E \rightarrow E_1 * E_2$	Si compatibles(E_1 .tipo, E_2 .tipo) Entonces E .dir = nuevaTemporal() E .tipo = max(E_1 .tipo, E_2 .tipo) α_1 = ampliar(E_1 .dir, E_1 .tipo, E .tipo) α_2 = ampliar(E_2 .dir, E_2 .tipo, E .tipo) genCode(E .dir '=' α_1 '**' α_2) Sino error(tipos incompatibles) Fin Si
$E \rightarrow E_1 / E_2$	Si compatibles(E_1 .tipo, E_2 .tipo) Entonces E .dir = nuevaTemporal() E .tipo = max(E_1 .tipo, E_2 .tipo)

	$\alpha_1 = \text{ampliar}(E_1.\text{dir}, E_1.\text{tipo}, E.\text{tipo})$ $\alpha_2 = \text{ampliar}(E_2.\text{dir}, E_2.\text{tipo}, E.\text{tipo})$ $\text{genCode}(E.\text{dir} = ' \alpha_1 ' / ' \alpha_2)$ Sino error(tipos incompatibles) Fin Si
$E \rightarrow E_1 \% E_2$	Si $E_1.\text{tipo} = E_2.\text{tipo} = \text{int}$ Entonces $E.\text{dir} = \text{nuevaTemporal}()$ $E.\text{tipo} = \text{int}$ $\text{genCode}(E.\text{dir} = ' E_1.\text{tipo} \% E_2.\text{tipo})$ Sino error(tipos incompatibles) Fin Si
$E \rightarrow E_1 E_2$	Si $\text{compatibles}(E_1.\text{tipo}, E_2.\text{tipo})$ Entonces $E.\text{dir} = \text{nuevaTemporal}()$ $E.\text{tipo} = \max(E_1.\text{tipo}, E_2.\text{tipo})$ $\alpha_1 = \text{ampliar}(E_1.\text{dir}, E_1.\text{tipo}, E.\text{tipo})$ $\alpha_2 = \text{ampliar}(E_2.\text{dir}, E_2.\text{tipo}, E.\text{tipo})$ $\text{genCode}(E.\text{dir} = ' \alpha_1 ' ' \alpha_2)$ Sino error(tipos incompatibles) Fin Si
$E \rightarrow E_1 \&\& E_2$	Si $\text{compatibles}(E_1.\text{tipo}, E_2.\text{tipo})$ Entonces $E.\text{dir} = \text{nuevaTemporal}()$ $E.\text{tipo} = \max(E_1.\text{tipo}, E_2.\text{tipo})$ $\alpha_1 = \text{ampliar}(E_1.\text{dir}, E_1.\text{tipo}, E.\text{tipo})$ $\alpha_2 = \text{ampliar}(E_2.\text{dir}, E_2.\text{tipo}, E.\text{tipo})$ $\text{genCode}(E.\text{dir} = ' \alpha_1 ' \&\& ' \alpha_2)$ Sino error(tipos incompatibles) Fin Si
$E \rightarrow E_1 > E_2$	Si $\text{compatibles}(E_1.\text{tipo}, E_2.\text{tipo})$ Entonces $E.\text{dir} = \text{nuevaTemporal}()$ $E.\text{tipo} = \max(E_1.\text{tipo}, E_2.\text{tipo})$ $\alpha_1 = \text{ampliar}(E_1.\text{dir}, E_1.\text{tipo}, E.\text{tipo})$ $\alpha_2 = \text{ampliar}(E_2.\text{dir}, E_2.\text{tipo}, E.\text{tipo})$ $\text{genCode}(E.\text{dir} = ' \alpha_1 ' > ' \alpha_2)$
$E \rightarrow E_1 < E_2$	Si $\text{compatibles}(E_1.\text{tipo}, E_2.\text{tipo})$ Entonces $E.\text{dir} = \text{nuevaTemporal}()$ $E.\text{tipo} = \max(E_1.\text{tipo}, E_2.\text{tipo})$ $\alpha_1 = \text{ampliar}(E_1.\text{dir}, E_1.\text{tipo}, E.\text{tipo})$ $\alpha_2 = \text{ampliar}(E_2.\text{dir}, E_2.\text{tipo}, E.\text{tipo})$ $\text{genCode}(E.\text{dir} = ' \alpha_1 ' < ' \alpha_2)$ Sino error(tipos incompatibles) Fin Si
$E \rightarrow E_1 \geq E_2$	Si $\text{compatibles}(E_1.\text{tipo}, E_2.\text{tipo})$ Entonces $E.\text{dir} = \text{nuevaTemporal}()$ $E.\text{tipo} = \max(E_1.\text{tipo}, E_2.\text{tipo})$ $\alpha_1 = \text{ampliar}(E_1.\text{dir}, E_1.\text{tipo}, E.\text{tipo})$ $\alpha_2 = \text{ampliar}(E_2.\text{dir}, E_2.\text{tipo}, E.\text{tipo})$ $\text{genCode}(E.\text{dir} = ' \alpha_1 ' \geq ' \alpha_2)$ Sino

	error(tipos incompatibles)
	Fin Si
$E \rightarrow E_1 \leq E_2$	Si compatibles(E_1 .tipo, E_2 .tipo) Entonces E.dir = nuevaTemporal() E.tipo = max(E_1 .tipo, E_2 .tipo) α_1 = ampliar(E_1 .dir, E_1 .tipo, E.tipo) α_2 = ampliar(E_2 .dir, E_2 .tipo, E.tipo) genCode(E.dir '=' α_1 '<=' α_2) Sino error(tipos incompatibles) Fin Si
$E \rightarrow E_1 \neq E_2$	Si compatibles(E_1 .tipo, E_2 .tipo) Entonces E.dir = nuevaTemporal() E.tipo = max(E_1 .tipo, E_2 .tipo) α_1 = ampliar(E_1 .dir, E_1 .tipo, E.tipo) α_2 = ampliar(E_2 .dir, E_2 .tipo, E.tipo) genCode(E.dir '=' α_1 '!=' α_2) Sino error(tipos incompatibles) Fin Si
$E \rightarrow E_1 == E_2$	Si compatibles(E_1 .tipo, E_2 .tipo) Entonces E.dir = nuevaTemporal() E.tipo = max(E_1 .tipo, E_2 .tipo) α_1 = ampliar(E_1 .dir, E_1 .tipo, E.tipo) α_2 = ampliar(E_2 .dir, E_2 .tipo, E.tipo) genCode(E.dir '=' α_1 '==' α_2) Sino error(tipos incompatibles) Fin Si
$E \rightarrow C$	E.dir = nuevaTemporal() E.tipo = C.tipo genCode(E.dir '=' C.base '(' C.dir ')')
$E \rightarrow \text{id}$	Si PilaTs.top().existe(id) Entonces E.tipo = PilaTs.top().getTipo(id) E.dir = id Sino Si PilaTs.bottom().existe(id) Entonces E.tipo = PilaTs.bottom().getTipo(id) E.dir = id Sino error("El id no fue declarado") Fin Si
$E \rightarrow \text{id}(N)$	Si PilaTs.bottom().existe(id) Entonces Si PilaTs.bottom().getCat() = 'func' Entonces Si PilaTs.bottom().getArgs() = N.lista Entonces E.dir = nuevaTemporal() E.tipo = PilaTs.bottom().getTipo(id) genCode(E.dir '=' 'call' id, N.lista.size) Sino error(La lista de parámetros no coincide con la lista de argumentos) Fin Si Sino error(El identificador no es una función) Fin Si

	Sino error("El id no fue declarado") Fin Si
$E \rightarrow Z$	E.dir = nuevaTemporal() E.tipo = Z.tipo genCode(E.dir '=' Z.base '[' Z.dir ']')
$E \rightarrow \text{num}$	E.tipo = num.tipo E.dir = num
$E \rightarrow \text{true}$	E.dir = 1 E.tipo = tablaTipos.getId(int)
$E \rightarrow \text{false}$	E.dir = 0 E.tipo = tablaTipos.getId(int)
$N \rightarrow M$	N.lista = M.lista
$N \rightarrow \varepsilon$	N.lista = nulo
$M \rightarrow M_1, E$	M.lista = M ₁ .lista M.lista.add(E.tipo) genCode('param' E.dir)
$M \rightarrow E$	M.lista = nuevaLista() M.lista.add(E.tipo) genCode('param' E.dir)
$C \rightarrow \text{id} [E]$	Si PilaTs.top().existe(id) Entonces tipo = PilaTs.top().getTipo(id) Si tablaTipos.getName(tipo) = 'array' Entonces Si E.tipo = int Entonces C.base = id C.tipo = tablaTipos.getTipoBase(tipo) C.tam = tablaTipos.getTam(C.tipo) C.dir = nuevaTemporal() genCode(C.dir '=' E.dir * C.tam) Sino error("El índice de un arreglo debe ser entero") Fin Si Sino error("El tipo del id no es un arreglo") Fin Si Sino Si PilaTs.bottom().existe(id) Entonces tipo = PilaTs.bottom().getTipo(id) Si tablaTipos.getName(tipo) = 'array' Entonces Si E.tipo = int Entonces C.base = id C.tipo = tablaTipos.getTipoBase(tipo) C.tam = tablaTipos.getTam(C.tipo) C.dir = nuevaTemporal() genCode(C.dir '=' E.dir * C.tam) Sino error("El índice de un arreglo debe ser entero") Fin Si Sino error("El tipo del id no es un arreglo") Fin Si

$C \rightarrow C_1 [E]$

```

Sino
    error("El id no fue declarado")
Fin Si

Si tablaTipos.getName(C1.tipo) = 'array' Entonces
    Si E.tipo = int Entonces
        C.base = C1.base
        C.tipo = tablaTipos.getTipoBase(C1.tipo)
        C.tam = tablaTipos.getTam(C.tipo)
        t = nuevaTemporal()
        C.dir = nuevaTemporal()
        genCode( t '=' E.dir '*' C.tam )
        genCode( C.dir '=' C1.dir '+' t )
    Sino
        error("El índice de un arreglo debe ser entero")
    Fin Si
Sino
    error("El arreglo no puede ser indexada tantas veces")
Fin Si

```

$Z \rightarrow Z_1 . id$

```

Si tablaTipos.getName(Z1.tipo)='struct' Entonces
    *ts = tablaTipos.getTS(Z1.tipo)
    Si *ts.existe(id) Entonces
        Z.dir = nuevaTemporal()
        genCode(Z.dir '=' Z1.dir + ts.getDir(id) )
        Z.base = Z1.base
        Z.tipo = ts.getTipo(id) )
    Sino
        error("El id no fue declarado")
    Fin Si
Sino
    error(No se puede usar el operador punto, el id no es un struct)
Fin Si

```

$Z \rightarrow id$

```

Si PilaTs.top().existe(id) Entonces
    Z.tipo = PilaTs.top().getTipo(id)
    Z.base = id
    Z.dir = 0
Sino Si PilaTs.bottom().existe(id) Entonces
    Z.tipo = PilaTs.bottom().getTipo(id)
    Z.base = id
    Z.dir = 0
Sino
    error("El id no fue declarado")
Fin Si

```

Realizar el proceso de compilación para el siguiente programa:

```

struct{
    int a;
    int b;
    float c;
}a, b;

```

```

float x, y;
int m, n;
int c;
int i, j;

```

```

float[5][7][3] w;
int z;
int w1;

def int suma(int x, int y){
    w1= 5;
    z = x + y + w1;
    return z;
}

def int main(){
    a.a= 8;
    a.b= 2;
    a.c = 1.5;
    m= 7;
    n = 15;
    c = suma(m, n);
    for(i=0; i < 5; i = i+1){
        for(j=0; j<7; j= j+1){
            for(k=0; k <3; k=k+1)
                a[i][j][k]= 0;
        }
    }

    for(i=0; i < 5; i = i+1){
        for(j=0; j<7; j= j+1){
            for(k=0; k <3; k=k+1)
                b[i][j][k]= b[i][j][k] + 3;
        }
    }
    return 0;
}

```

Mediante la definición dirigida por sintaxis, es decir; siguiendo las reglas gramaticales junto con las reglas semánticas genere los siguientes datos:

1. El grafo de dependencias
2. La pila de tablas de símbolos.
3. La tabla de tipos.
4. El código intermedio

Sugerencia: trabaje el árbol en una hoja doble carta.